Low-Level Network Packet Inspection & Protocol Analysis

Task 3: Network Traffic Analysis Lab

Prepared By:

Daaniya Absar

Date: August 31, 2026

Target: 192.168.56.101 (Metasploitable 2)

Tools: Wireshark, tcpdump

ProGree Internship Program

Confidential – For Internal Review Only

Contents

1 Executive Summary

1.1 Overview

This report documents the findings of Task 3: Low-Level Network Packet Inspection & Protocol Analysis. The objective was to monitor data packet streams, analyze TCP handshake protocols, track individual packet stream data layers, isolate unauthorized data exfiltration trends, and identify unencrypted protocol vulnerability configurations.

The assessment was conducted against the target system **192.168.56.101** (Metasploitable 2) from the attacker machine **192.168.56.102** (Kali Linux). Network traffic was captured and analyzed using **Wireshark**, **tcpdump**, and **tshark**.

1.2 Key Findings

The analysis revealed critical security vulnerabilities in clear-text protocols:

Protocol	Port	Encryption	Security Status
FTP	21	NONE	CRITICAL - Credentials captured
HTTP	80	NONE	CRITICAL - Clear-text communication
Telnet	23	NONE	CRITICAL - Complete session visible
SSH	22	YES	SECURE - Encrypted

1.3 Captured Credentials

The following credentials were successfully intercepted:

Protocol	Credential	Visibility
FTP	`msfadmin / msfadmin`	Clear-text
HTTP	`admin / admin123`	Clear-text (POST data)
Telnet	`msfadmin / msfadmin`	Clear-text
SSH	`msfadmin / msfadmin`	NOT VISIBLE (Encrypted)

1.4 Risk Impact

If exploited, these vulnerabilities could allow attackers to:

- Capture usernames and passwords in clear-text
- Intercept sensitive data transmissions
- Hijack sessions and impersonate users

- Gain unauthorized access to systems

1.5 Recommendations

Based on the findings, the following high-level recommendations are made:

1. Immediately disable FTP, HTTP, and Telnet
2. Replace with SFTP/FTPS, HTTPS, and SSH
3. Implement encryption for all network communications
4. Update outdated software (Apache, PHP)

2 Assessment Methodology

2.1 Scope

The assessment was conducted with the following parameters:

- **Target System:** 192.168.56.101 (Metasploitable 2)
- **Attacker System:** 192.168.56.102 (Kali Linux)
- **Network:** Isolated host-only network (192.168.56.0/24)
- **Assessment Type:** Network packet inspection and protocol analysis
- **Assessment Date:** August 31, 2026
- **Tools Used:** Wireshark, tcpdump, tshark, curl, ftp, telnet, ssh

2.2 Tools Used

Tool	Purpose
Wireshark	Graphical packet capture and analysis
tcpdump	Command-line packet capture
tshark	Command-line packet analysis
curl	Generate HTTP traffic
ftp	Generate FTP traffic
telnet	Generate Telnet traffic
ssh	Generate SSH traffic (encrypted comparison)

2.3 Assessment Phases

Phase 1: Traffic Generation

Network traffic was generated using various protocols to simulate real-world communication patterns. This included FTP file transfers, HTTP web requests, Telnet remote sessions, and SSH encrypted connections.

Phase 2: Packet Capture

Traffic was captured using tcpdump and Wireshark. Separate capture files were created for each protocol to facilitate focused analysis.

Phase 3: Protocol Analysis

Each capture was analyzed to identify:

- Clear-text credentials and data
- Protocol vulnerabilities
- TCP handshake patterns

- Data exfiltration indicators

Phase 4: Security Comparison

Encrypted protocols (SSH) were compared with clear-text protocols (FTP, HTTP, Telnet) to demonstrate the importance of encryption.

Phase 5: Reporting

Findings were documented, including captured credentials, vulnerabilities identified, and recommendations for remediation.

2.4 Testing Limitations

- Testing was limited to network-layer analysis
- Only TCP-based protocols were analyzed
- The assessment was conducted in an isolated lab environment

3 Traffic Generation

3.1 Commands Executed

3.1.1 FTP Traffic

```
# Connect to FTP
ftp 192.168.56.101
# Login: msfadmin/msfadmin
ls -la
pwd
bye
```

3.1.2 HTTP Traffic

```
# Generate HTTP traffic
curl http://192.168.56.101/
curl http://192.168.56.101/phpinfo.php
curl -X TRACE http://192.168.56.101/
curl -X POST -d "username=admin&password=admin123" \
    http://192.168.56.101/mutillidae/index.php?page=login.php
```

3.1.3 Telnet Traffic

```
# Connect via Telnet
telnet 192.168.56.101
# Login: msfadmin/msfadmin
whoami
id
pwd
exit
```

3.1.4 SSH Traffic (Encrypted Comparison)

```
# Connect via SSH (encrypted)
ssh msfadmin@192.168.56.101
# Password: msfadmin
whoami
id
exit
```

3.2 Packet Capture Commands

```
# Capture HTTP traffic
sudo tcpdump -i eth0 port 80 -w http_traffic.pcap

# Capture Telnet traffic
```

```
sudo tcpdump -i eth0 port 23 -w telnet_traffic.pcap

# Capture SSH traffic
sudo tcpdump -i eth0 port 22 -w ssh_traffic.pcap
```

4 FTP Protocol Analysis

4.1 Overview

FTP (File Transfer Protocol) is a clear-text protocol that transmits all data, including authentication credentials, without encryption.

4.2 Captured Evidence

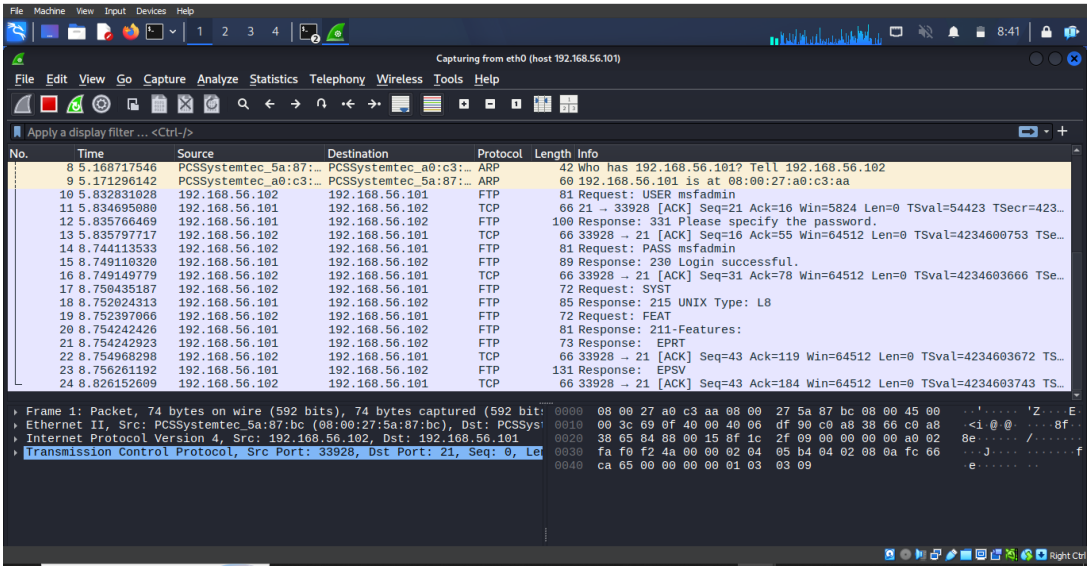

Figure 1: FTP Traffic Analysis - Clear-text Credentials Captured

4.3 Captured Credentials

Field	Value
Username	`msfadmin`
Password	`msfadmin`
Protocol	FTP (Port 21)
Visibility	Clear-text

4.4 Security Issues Identified

- **Credentials in Clear-text:** Username and password transmitted without encryption
- **Commands Visible:** All FTP commands visible to packet sniffers
- **Data Transfers Visible:** File contents transmitted unencrypted
- **Vulnerable to MITM:** Susceptible to man-in-the-middle attacks

4.5 Remediation

- Disable FTP immediately
- Replace with SFTP (SSH File Transfer Protocol)

- Replace with FTPS (FTP over SSL/TLS)
- Use SSH for secure file transfers

5 HTTP Protocol Analysis

5.1 Overview

HTTP (Hypertext Transfer Protocol) is a clear-text protocol that transmits all web communications without encryption.

5.2 Captured Evidence

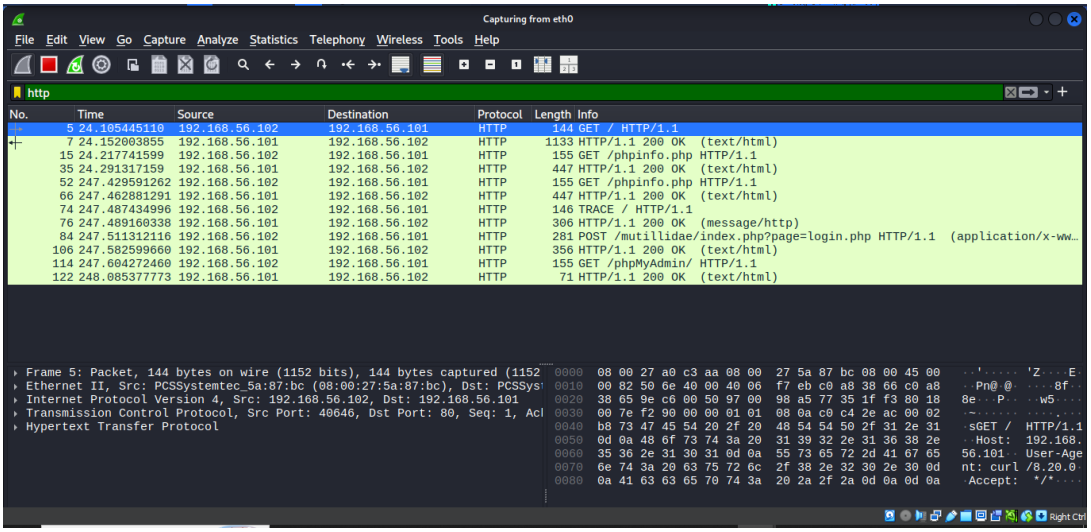

Figure 2: HTTP Traffic Analysis - Clear-text Communication and Vulnerabilities

5.3 Captured Data

Data Type	Value
HTTP Method	GET /phpinfo.php
User-Agent	curl/8.20.0
Host	192.168.56.101
POST Data	username=admin&password=admin123
TRACE Method	Enabled (Vulnerable to XST)

5.4 Vulnerabilities Identified

5.4.1 phpinfo.php Exposed

The `phpinfo.php` file reveals sensitive system information:

- PHP Version: 5.2.4-2ubuntu5.10
- System: Linux metasploitable 2.6.24-16-server
- Server API: CGI/FastCGI
- Apache Version: 2.2.8 (Ubuntu)
- Server paths and environment variables

5.4.2 TRACE Method Enabled

The HTTP TRACE method is enabled, allowing:

- Cross-Site Tracing (XST) attacks
- Cookie theft via XSS
- Session hijacking

5.4.3 Clear-text POST Data

Login credentials transmitted in clear-text:

- `username=admin`
- `password=admin123`

5.4.4 Outdated Software

- Apache 2.2.8 (released 2008 - 16+ years old)
- PHP 5.2.4 (released 2007 - 17+ years old)

5.5 Remediation

- Disable HTTP, use HTTPS with SSL/TLS certificate
- Remove phpinfo.php: `sudo rm /var/www/phpinfo.php`
- Disable TRACE method: `TraceEnable Off` in Apache config
- Upgrade Apache to 2.4.x and PHP to 8.x
- Implement security headers (CSP, HSTS)

6 Telnet Protocol Analysis

6.1 Overview

Telnet is a clear-text remote access protocol that transmits all session data, including credentials, without encryption.

6.2 Captured Data

Data Type	Value
Login Prompt	`metasploitable login:`
Username	`msfadmin`
Password Prompt	`Password:`
Password	`msfadmin`
Command	`whoami`
Output	`msfadmin`
Command	`id`
Output	`uid=1000(msfadmin) gid=1000(msfadmin)`
Command	`pwd`
Output	`/home/msfadmin`

6.3 Security Issues Identified

- **Complete Session Visible:** Entire Telnet session captured
- **Credentials Exposed:** Username and password in clear-text
- **Commands Visible:** All commands and responses captured
- **System Information:** System information exposed
- **No Encryption:** Zero protection against interception

6.4 Remediation

- Disable Telnet immediately
- `sudo systemctl stop telnetd`
- Use SSH exclusively for remote access
- All SSH traffic is encrypted and secure

7 SSH Protocol Analysis

7.1 Overview

SSH (Secure Shell) is an encrypted protocol that provides secure remote access and data transmission.

7.2 Captured Evidence

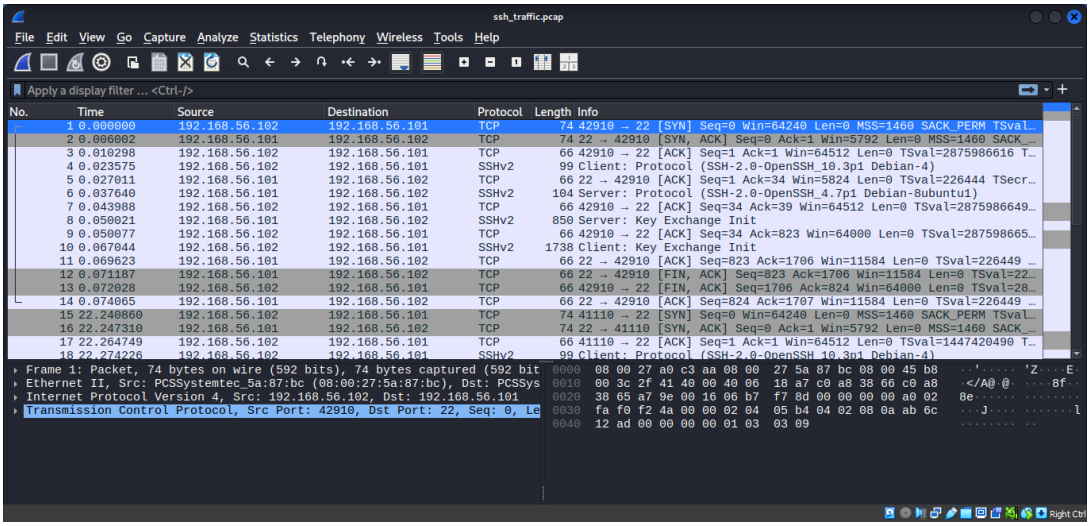

Figure 3: SSH Traffic Analysis - Encrypted Communication

7.3 Analysis Results

Data Type	Visibility
Username	NOT VISIBLE (Encrypted)
Password	NOT VISIBLE (Encrypted)
Commands	NOT VISIBLE (Encrypted)
Output	NOT VISIBLE (Encrypted)
File Transfers	NOT VISIBLE (Encrypted)

7.4 Security Comparison

Protocol	Encryption	Credentials Visible	Security
FTP	NONE	YES	INSECURE
HTTP	NONE	YES	INSECURE
Telnet	NONE	YES	INSECURE
SSH	YES	NO	SECURE

7.5 Why SSH is Secure

- All data is encrypted using strong cryptography
- No credentials transmitted in clear-text
- Commands and output protected from interception
- Immune to packet sniffing attacks
- Provides authentication and integrity checking

7.6 Recommendation

- Use SSH for ALL remote access
- Disable Telnet and FTP
- Implement SSH key-based authentication
- Regular SSH security audits

8 Real-World Impact of Packet Analysis

8.1 Why This Matters

Packet analysis is not just an academic exercise. It is a critical cybersecurity capability that has prevented (or could have prevented) some of the largest data breaches in history.

8.2 Case Study 1: Target Data Breach (2013)

What Happened	How Packet Analysis Would Have Helped
Hackers stole 40 million credit cards	Packet analysis would have shown data leaving the network
Attack went undetected for weeks	Would have seen unusual outbound traffic patterns
Cost: $200+ million	Early detection could have prevented this

8.2.1 What Went Wrong

In 2013, Target Corporation suffered a massive data breach where hackers gained access to the company's network through a third-party HVAC vendor. Once inside, they installed malware on point-of-sale (POS) systems that captured credit card data from 40 million customers.

8.2.2 How Packet Analysis Would Have Helped

- **Data Exfiltration Detection:** Packet analysis would have detected large volumes of data leaving the network
- **Anomaly Detection:** Unusual traffic patterns at odd hours would have been flagged
- **Unauthorized Communication:** Communication with external command-and-control servers would have been visible

8.2.3 Lesson Learned

If Target had monitored their network traffic, they would have detected the breach immediately instead of months later.

8.3 Case Study 2: Marriott Data Breach (2018)

What Happened	How Packet Analysis Would Have Helped
500 million guest records stolen	Packet analysis would have shown massive data transfers
Attackers inside network for 4 years	Would have detected anomalies in traffic patterns
Cost: $124 million fine	Early detection could have prevented this

8.3.1 What Went Wrong

Marriott International discovered in 2018 that hackers had been inside their network since 2014. The attackers stole personal information from up to 500 million guests, including names, addresses, passport numbers, and credit card details.

8.3.2 How Packet Analysis Would Have Helped

- **Prolonged Access Detection:** Unusual persistent connections over 4 years would have been detected
- **Data Exfiltration:** Massive data transfers would have been visible
- **Unauthorized Access:** Communication patterns from internal systems to external IPs would have been flagged

8.3.3 Lesson Learned

4 years of undetected access could have been caught with proper network monitoring. Packet analysis is essential for early breach detection.

8.4 What You Learned in This Lab

Your Action	What You Found	Real-World Threat
Captured FTP traffic	Username and password in clear-text	Hackers steal credentials this way
Captured HTTP traffic	phpinfo.php exposes system info	Attackers gather intelligence
Captured HTTP traffic	TRACE method enabled	Attackers steal cookies via XST
Captured Telnet traffic	Complete session visible	Attackers hijack sessions
Captured SSH traffic	Everything encrypted	This is how you stay safe!

9 TCP Handshake Analysis

9.1 Overview

The TCP 3-way handshake establishes a connection between client and server.

9.2 FTP Handshake

Step	Source	Destination	Flags
1	192.168.56.102	192.168.56.101	SYN (Client: "I want to connect")
2	192.168.56.101	192.168.56.102	SYN-ACK (Server: "OK, I'm ready")
3	192.168.56.102	192.168.56.101	ACK (Client: "Connection established")
4	192.168.56.102	192.168.56.101	DATA (FTP Login: USER msfadmin)

9.3 HTTP Handshake

Step	Source	Destination	Flags
1	192.168.56.102	192.168.56.101	SYN
2	192.168.56.101	192.168.56.102	SYN-ACK
3	192.168.56.102	192.168.56.101	ACK
4	192.168.56.102	192.168.56.101	DATA (HTTP GET /)

9.4 TCP Flags

Flag	Meaning
SYN	Initiates connection
ACK	Acknowledges receipt
SYN-ACK	Server accepts connection
RST	Aborts connection
FIN	Gracefully closes connection

10 Security Recommendations

10.1 Priority 1 – Immediate (24-72 Hours)

Action	Implementation
Disable FTP	`sudo systemctl stop vsftpd`
Disable HTTP	Use HTTPS with SSL/TLS certificate
Disable Telnet	`sudo systemctl stop telnetd`
Use SSH exclusively	Replace Telnet with SSH
Disable TRACE method	`TraceEnable Off` in Apache config
Remove phpinfo.php	`sudo rm /var/www/phpinfo.php`

10.2 Priority 2 – Short Term (7-10 Days)

Action	Implementation
Upgrade Apache	`sudo apt update && sudo apt upgrade apache2`
Upgrade PHP	`sudo apt update && sudo apt upgrade php`
Implement HTTPS	Configure SSL/TLS certificate
Remove test files	Remove /test/, /doc/ directories
Implement security headers	CSP, HSTS, X-Content-Type-Options

10.3 Priority 3 – Long Term (30 Days)

Action	Implementation
Network Segmentation	Isolate vulnerable systems
Firewall Configuration	Block unnecessary ports
Monitoring Solution	Implement IDS/IPS
Security Audits	Regular vulnerability scanning
Training	Security awareness for staff

10.4 Secure Protocol Checklist

FTP disabled → Use SFTP or FTPS

HTTP disabled → Use HTTPS

Telnet disabled → Use SSH

TRACE method disabled

phpinfo.php removed

Security headers implemented

Software updated to latest versions

Regular security assessments scheduled

11 Appendices

11.1 Appendix A: Complete Evidence Summary

Protocol	Credentials	Encryption	Status
FTP	msfadmin/msfadmin	NONE	VULNERABLE
HTTP	admin/admin123	NONE	VULNERABLE
Telnet	msfadmin/msfadmin	NONE	VULNERABLE
SSH	msfadmin/msfadmin	YES	SECURE

11.2 Appendix B: Command Reference

```
# Start Wireshark
sudo wireshark

# Capture FTP traffic
sudo tcpdump -i eth0 port 21 -w ftp_traffic.pcap

# Capture HTTP traffic
sudo tcpdump -i eth0 port 80 -w http_traffic.pcap

# Capture Telnet traffic
sudo tcpdump -i eth0 port 23 -w telnet_traffic.pcap

# Capture SSH traffic
sudo tcpdump -i eth0 port 22 -w ssh_traffic.pcap

# Extract FTP credentials
tshark -r ftp_traffic.pcap -Y "ftp.request.command == USER || ftp.
    request.command == PASS" -T fields -e ftp.request.command -e ftp.
    request.arg

# Extract HTTP requests
tshark -r http_traffic.pcap -Y "http.request" -T fields -e http.request
    .method -e http.request.uri

# View TCP handshake
tshark -r capture.pcap -Y "tcp.flags.syn == 1 || tcp.flags.ack == 1" -T
     fields -e ip.src -e ip.dst -e tcp.flags
```

11.3 Appendix C: Wireshark Filters Used

Filter	Purpose
`ftp`	View all FTP traffic
`ftp.request.command == "USER"`	Find FTP username
`ftp.request.command == "PASS"`	Find FTP password
`http`	View all HTTP traffic
`http.request.method == "GET"`	View HTTP GET requests
`http.request.method == "TRACE"`	Find TRACE method
`telnet`	View all Telnet traffic
`ssh`	View SSH traffic (encrypted)
`tcp.flags.syn == 1`	View TCP SYN packets
`tcp.flags.ack == 1`	View TCP ACK packets

11.4 Appendix D: Risk Matrix

Priority	Timeline	Action Required
P1 – Critical	24-72 hours	Immediate fix required
P2 – High	7-10 days	Scheduled upgrade
P3 – Medium	30 days	Planned remediation
P4 – Low	Ongoing	Monitor and maintain

Thank You

Prepared with diligence by

Daaniya Absar

ProGree Internship Program

Confidential – For Internal Review Only